

LegalEase

AI-Powered Legal Document Generator

Phase 3: Project Design

Project Report

1. Purpose of This Phase

The Design phase translates the requirements gathered earlier into a concrete technical blueprint: system architecture, data design, interface design, and module structure that developers will implement in the next phase.

2. System Architecture Overview

LegalEase follows a layered, client-server architecture consisting of four main layers: the Presentation Layer (web front end), the Application Layer (backend API and business logic), the AI/NLP Layer (document generation engine), and the Data Layer (database and template store).

- **Presentation Layer:** A responsive web interface where users select a template, provide input, and review/edit generated documents.
- **Application Layer:** REST API handling authentication, request routing, business rules, and clause validation logic.
- **AI/NLP Layer:** Processes natural-language input, maps it to template variables, and generates/refines clause text using a language model.
- **Data Layer:** Stores user accounts, document templates, clause libraries, and generated document history.

3. High-Level Architecture Diagram (Description)

User Browser → Web Front End → API Gateway → Application Server (Business Logic) → AI/NLP Generation Service ↔ Template & Clause Repository → Database (User, Document, Template tables) → Generated Document returned to Front End for review and export.

4. Module Design

Module	Responsibility
Authentication Module	User registration, login, session/token management
Template Manager	Stores and serves document templates by category
Input Processor	Parses natural-language input into structured fields
AI Generation Engine	Generates draft clauses/document text using NLP model
Clause Validator	Checks for mandatory clauses and flags missing items
Editor Module	Provides in-browser editing of the generated draft
Export Module	Converts final document to PDF/Word

Module	Responsibility
History Module	Stores and retrieves user's past documents

5. Database Design (Key Entities)

Entity	Key Attributes
User	user_id, name, email, password_hash, created_at
DocumentTemplate	template_id, category, title, base_structure, jurisdiction
Clause	clause_id, template_id, clause_text, is_mandatory
GeneratedDocument	doc_id, user_id, template_id, content, created_at, status
InputSession	session_id, user_id, raw_input, extracted_fields

6. Entity Relationship Overview

A User can create many GeneratedDocuments (one-to-many). Each GeneratedDocument is based on one DocumentTemplate, while a DocumentTemplate can be reused across many GeneratedDocuments (one-to-many). Each DocumentTemplate is composed of multiple Clauses (one-to-many). Each InputSession is linked to one User and produces one GeneratedDocument.

7. UML Use Case Diagram (Description)

Actor: **User**. Use cases: Register/Login, Select Template, Provide Input, Generate Draft, Edit Document, Validate Clauses, Export Document, View History. Actor: **Admin** (secondary). Use cases: Manage Templates, Manage Clause Library.

8. Sequence Flow: Document Generation

- 1 User logs in and selects a document category.
- 2 User enters requirement description in natural language.
- 3 Input Processor extracts structured fields from the description.
- 4 AI Generation Engine combines extracted fields with the base template to produce draft text.
- 5 Clause Validator checks the draft against the mandatory clause list.
- 6 System returns the draft to the front end for user review.
- 7 User edits the draft as needed and requests export.

8 Export Module converts the final content to PDF/Word and returns it for download.

9. User Interface Design (Wireframe Description)

- **Dashboard:** Displays document categories as cards and a shortcut to "My Documents".
- **Input Screen:** A text area for natural language description plus optional guided form fields.
- **Draft Review Screen:** A rich-text editor showing the generated document with highlighted clauses flagged for review.
- **Export Screen:** Options to download as PDF/Word or request e-signature.
- **History Screen:** A table listing past documents with date, type, and download links.

10. Technology Stack Selected

Layer	Technology
Frontend	React.js, HTML5, CSS3 / Tailwind
Backend	Node.js / Python (Flask or FastAPI)
AI/NLP Engine	Large Language Model API + rule-based clause templates
Database	PostgreSQL / MongoDB
Export	PDF generation library (e.g., ReportLab) and DOCX library
Hosting	Cloud platform (e.g., AWS / Render / Vercel)

11. Security Design Considerations

- Passwords stored using a strong one-way hashing algorithm rather than plain text.
- API endpoints protected using token-based authentication (e.g., JWT) with expiry.
- Input sanitization on the Input Processor to prevent injection through user-provided text.
- Role separation between regular Users and Admin accounts for template management.
- Encrypted connections (HTTPS) between client, server, and the AI generation service.

12. Error Handling Design

The system is designed to handle failure gracefully at each stage of the generation pipeline: invalid or incomplete input triggers a request for clarification rather than a failed draft; AI service timeouts fall back to a template-only draft with a notice to the user; and export failures present a retry option without losing the user's edited content.

13. Outcome of This Phase

This phase delivers the architecture diagram, database schema, module breakdown, UML descriptions, security considerations, error-handling strategy, and UI wireframes needed to move confidently into project planning and, subsequently, development.